

梯度下降算法改进

学习目标

优化遇到的问题

梯度消失

局部优化

参数初始化策略（复习）

批梯度下降算法（Batch Gradient Descent）

Mini——Batch Gradient Descent

批梯度下降与Mini——Batch梯度下降的区别

梯度下降优化影响

大小选择

指数加权平均

动量梯度下降法

RMSProp算法

Adam算法

TensorFlow Adam算法API

学习率衰减

其它非算法优化的方式-标准化输入

学习目标

- 目标
 - 了解深度学习遇到的一些问题
 - 知道批梯度下降与MiniBatch梯度下降的区别
 - 知道指数加权平均的意义
 - 知道动量梯度、RMSProp、Adam算法的公式意义
 - 知道学习率衰减方式
 - 知道参数初始化策略的意义
- 应用
 - 无

深度学习难以在大数据领域发挥最大效果的一个原因是，在巨大的数据集基础上进行训练速度很慢。而优化算法能够帮助我们快速训练模型，提高计算效率。接下来我就去看有哪些方法能够解决我们刚才

遇到的问题或者类似的问题。

优化遇到的问题

- 梯度消失
- 局部优化

梯度消失

在梯度函数上出现的以指数级递增或者递减的情况分别称为梯度爆炸或者梯度消失。

假设 $g(z) = z, b^{[l]} = 0$, 对于目标输出有: $\hat{y} = W^{[L]}W^{[L-1]}...W^{[2]}W^{[1]}X$

- 对于 $W^{[l]}$ 的值大于 1 的情况, 激活函数的值将以指数级递增;
- 对于 $W^{[l]}$ 的值小于 1 的情况, 激活函数的值将以指数级递减。

在计算梯度时, 根据不同情况梯度函数也会以指数递增或递减, 导致训练导致难度上升, 梯度下降算法的步长会变得非常小, 需要训练的时间将会非常长。

2.2.1.1 梯度消失

$$\begin{matrix} 10, 10^2, 10^3, 10^4 = \\ 0.1, 0.1^2, 0.1^3, 0.1^4 \end{matrix}$$
$$w^{[1]} * \dots * w^{[2]} * w^{[1]}^T x$$

最后一层输出: 非常大
最后一层输出: 非常小

在梯度函数上出现的以指数级递增或者递减的情况分别称为**梯度爆炸**或者**梯度消失**。

假设 $g(z) = z, b^{[l]} = 0$,对于目标输出有: $\hat{y} = W^{[L]}W^{[L-1]}...W^{[2]}W^{[1]}X$

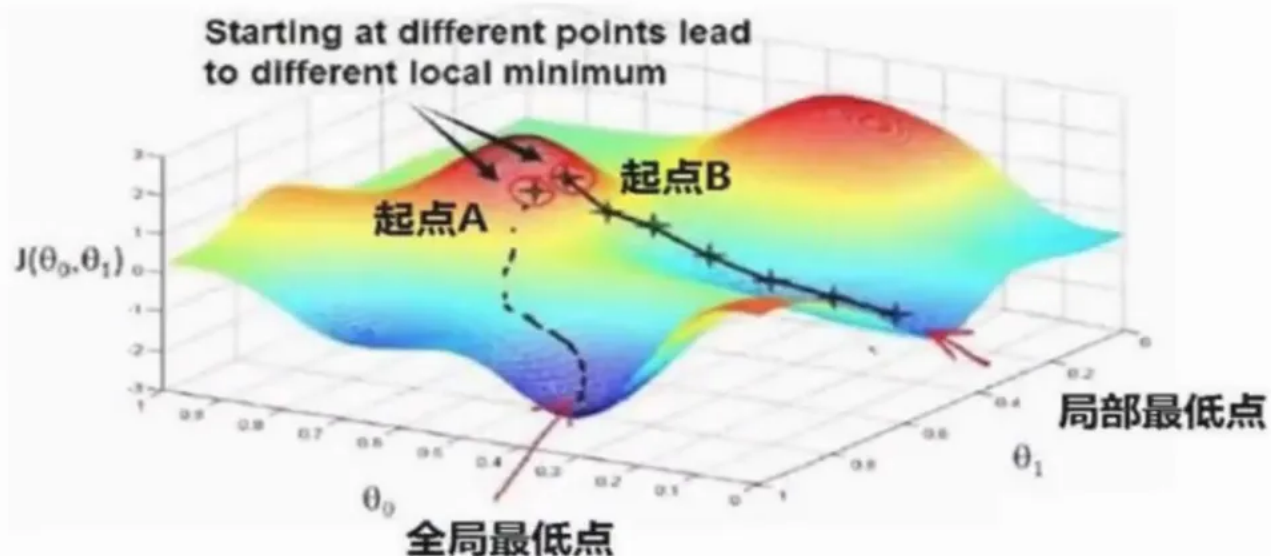
- 对于 $W^{[l]}$ 的值大于 1 的情况, 激活函数的值将以指数级递增; x
- 对于 $W^{[l]}$ 的值小于 1 的情况, 激活函数的值将以指数级递减。

没加激活, 偏置默认为0

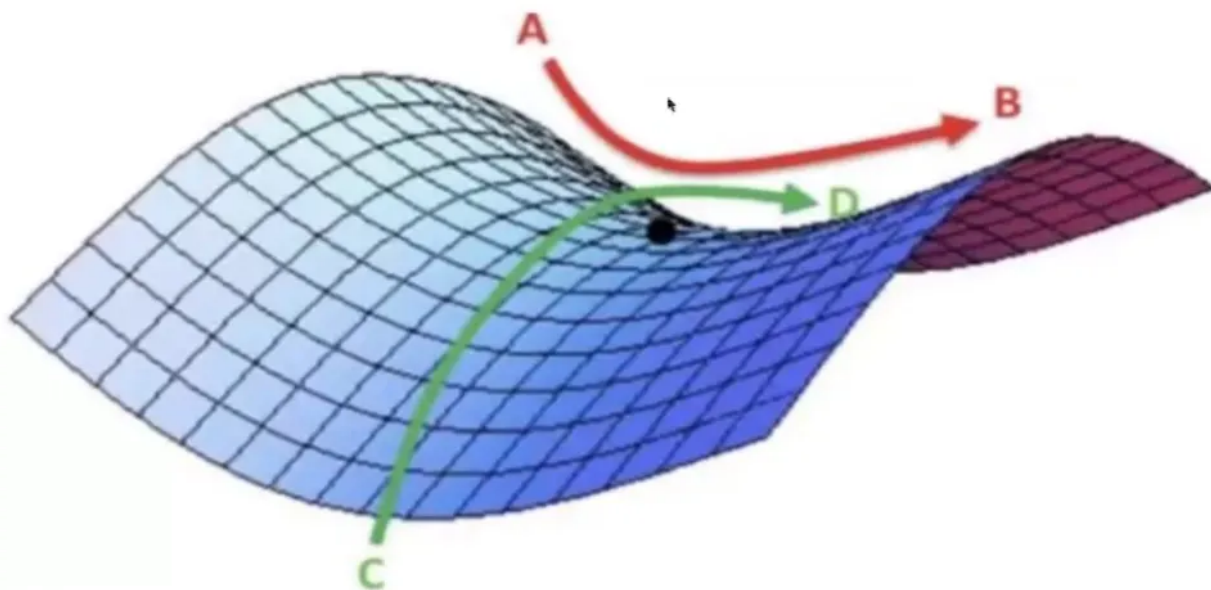


在计算梯度时, 根据不同情况梯度函数也会以指数级递增或递减, 导致训练导致难度上升, 梯度下降算法的步长会变得非常小, 需要训练的时间将会非常长。

局部优化



鞍点是函数上的导数为零，但不是轴上局部极值的点。通常梯度为零的点是上图所示的鞍点，而非局部最小值。减少损失的难度也来自误差曲面中的鞍点，而不是局部最低点。



- 在训练较大的神经网络、存在大量参数，并且成本函数被定义在较高的维度空间时，困在极差的局部最优基本不会发生
- 鞍点附近的平稳段会使得学习非常的缓慢，而这也是需要后面的动量梯度下降法、RMSProp以及Adam优化算法能够加速学习的原因，他们能帮助尽早走出平稳段。

解决办法有多种形式，通常会结合一些形式一起进行

- 初始化参数策略（第一部分第四节提到）
- Mini梯度下降法
- 梯度下降算法的优化

- 学习率衰减

参数初始化策略（复习）

由于在 $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$ 公式中，当输入的数量 n 较大时，如果每个 w_i 的值都小一些，这样它们的和得到的 z 也会非常大，所以会造成我们之前在第一部分最后一节当中介绍的。所以都会初始化比较小的值。

批梯度下降算法（Batch Gradient Descent）

- 定义：批梯度下降法，即同时处理整个训练集

其在更新参数时使用所有的样本来进行更新。对整个训练集进行梯度下降法的时候，我们必须处理整个训练数据集，然后才能进行一步梯度下降，即每一步梯度下降法需要对整个训练集进行一次处理，如果训练数据集很大的时候，处理速度就会比较慢。

所以换一种方式，每次处理训练数据的一部分进行梯度下降法，则我们的算法速度会执行的更快。

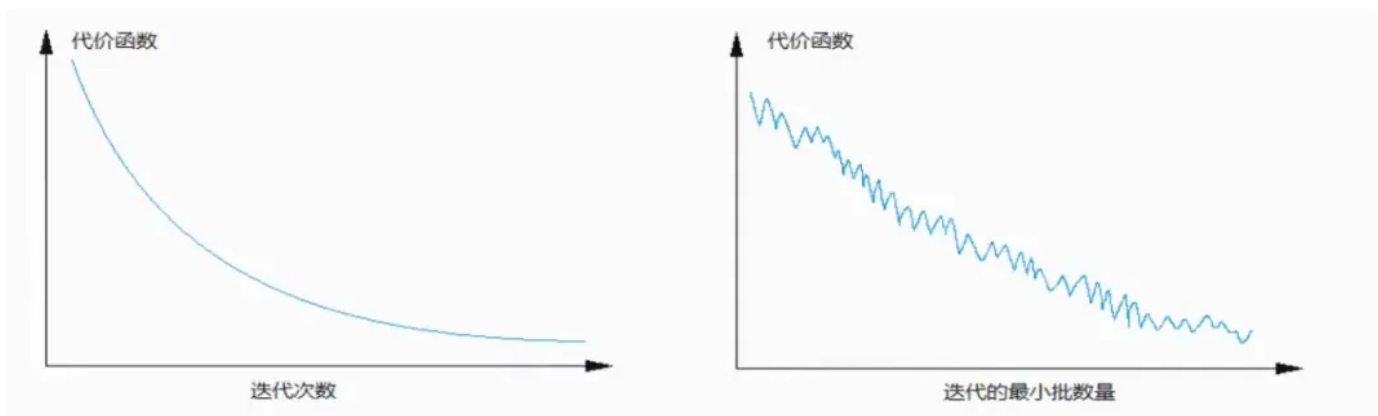
Mini——Batch Gradient Descent

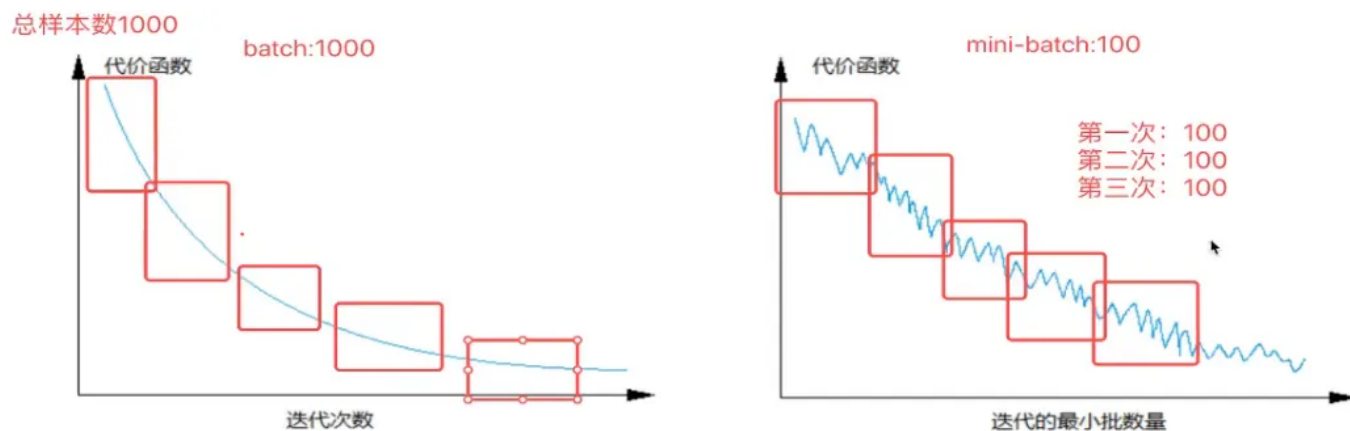
定义：Mini——Batch梯度下降法（小批量梯度下降法）每次同时处理固定大小的数据集。

种类：Mini——Batch的大小为1，即随机梯度下降法（sgd），使用Mini——Batch梯度下降法，对整个训练集的一次遍历只做Mini——Batch个样本的梯度下降，一直循环整个训练集。

批梯度下降与Mini——Batch梯度下降的区别

batch梯度下降法和Mini——Batch梯度下降法代价函数的变化趋势如下：

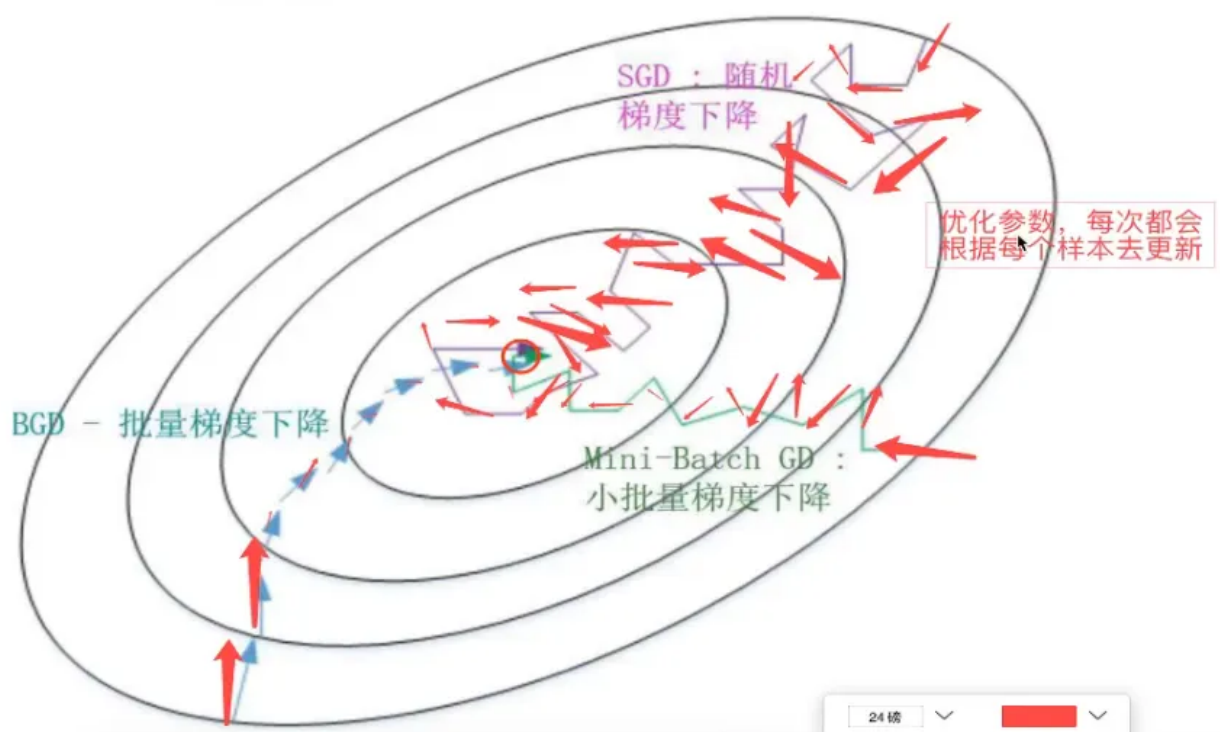
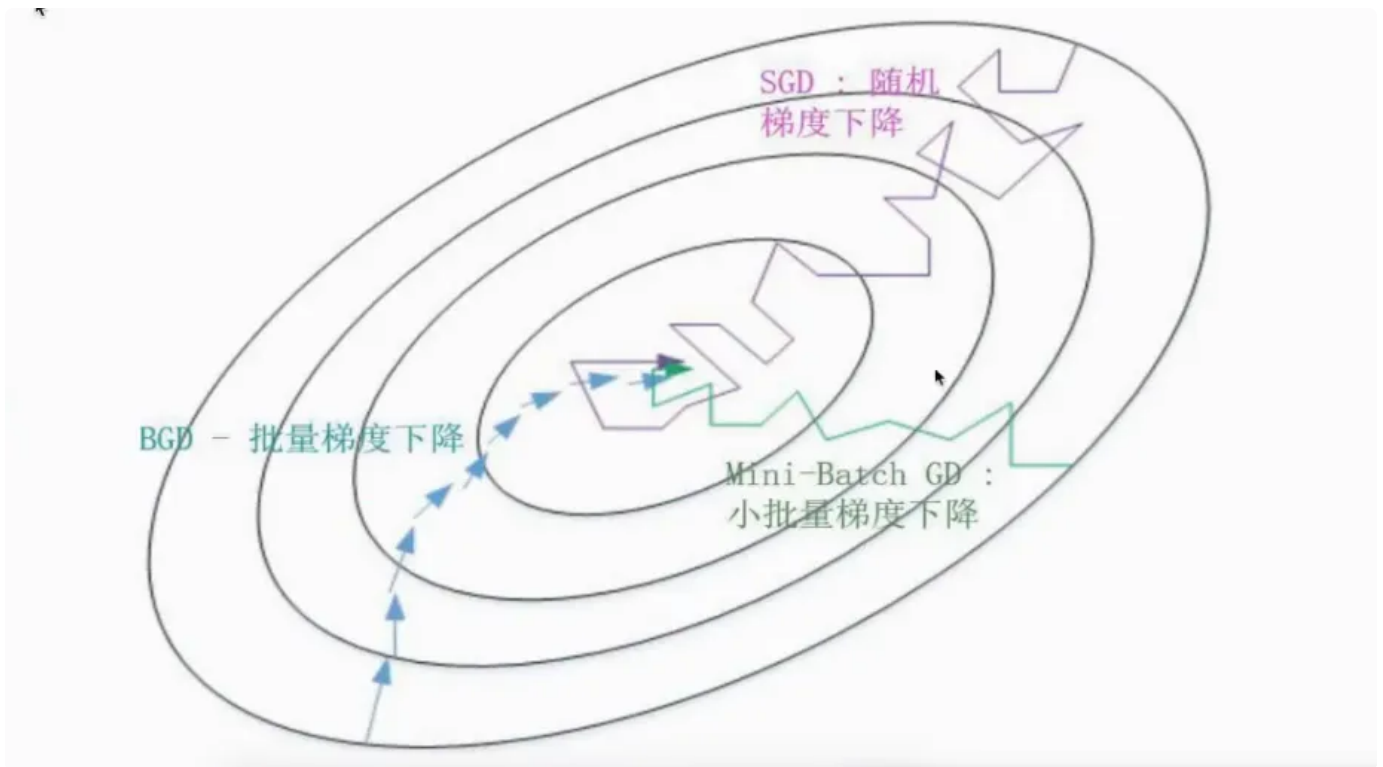




那么对于梯度下降优化带来的影响

梯度下降优化影响

- batch梯度下降法：
 - 对所有 m 个训练样本执行一次梯度下降，每一次迭代时间较长，训练过程慢；
 - 相对噪声低一些，成本函数总是向减小的方向下降。
- 随机梯度下降法
 - 对每一个训练样本执行一次梯度下降，训练速度快，但丢失了向量化带来的计算加速；
 - 有很多噪声，需要适当减小学习率，成本函数总体趋势向全局最小值靠近，但永远不会收敛，而是一直在最小值附近波动。



因此，选择一个合适大小进行mini-batch 梯度下降，可以实现快速学习，也应用了向量化带来的好处，且成本函数的下降处于前两者之间。

大小选择

- 如果训练样本的大小比较小，如 $m \leq 2000$ 时，选择 batch 梯度下降法；
- 如果训练样本的大小比较大，选择 Mini-Batch 梯度下降法。为了和计算机的信息存储方式相适应，代码在 mini-batch 大小为 2 的幂次时运行要快一些。典型的大小为 $2^6, 2^7, 2^8, 2^9$, mini-batch 的大小要符合 CPU/GPU 内存。

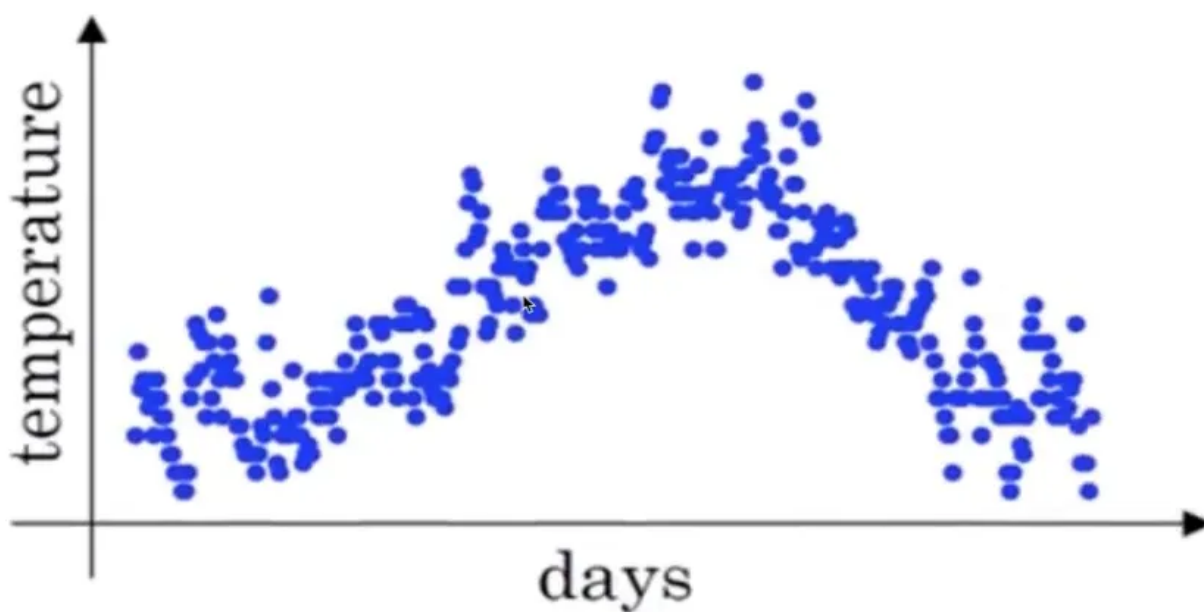
需要根据经验快速尝试，找到能够最有效地减少成本函数的值

那么第二种方式是通过优化梯度下降过程，会比梯度下降算法的速度更快些

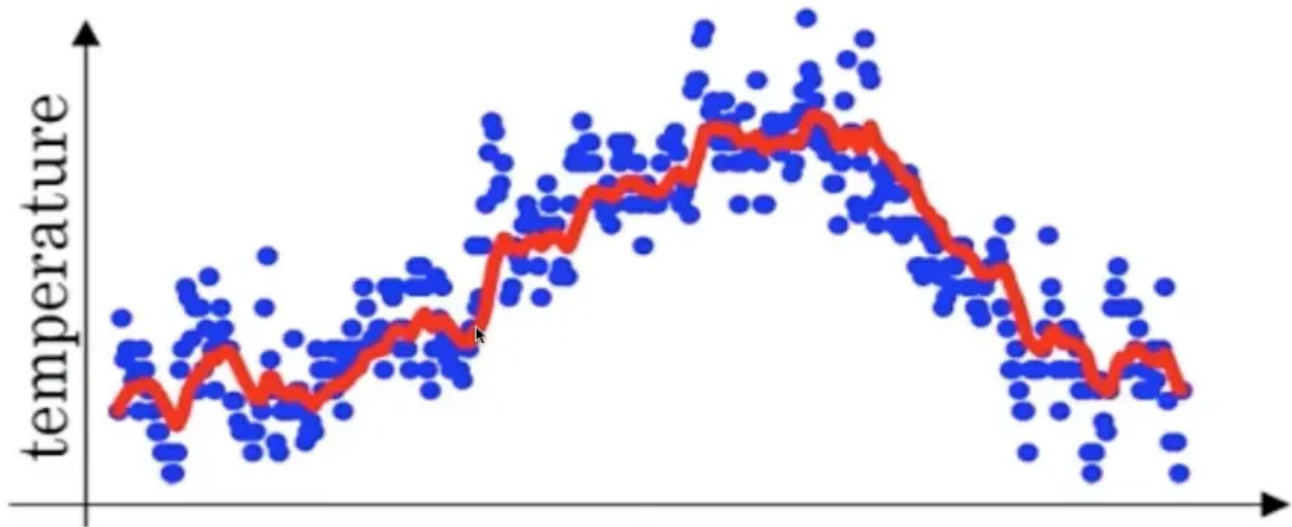
指数加权平均

指数加权平均是一种常用的序列数据处理方式。通常用在序列场景如金融序列分析、温度变化序列分析。

假设给定一个序列，假如北京一年每天的气温值，图中蓝色的点代表真实数据



那么这样的气温值变化可以理解成优化的过程波动较大，异常较多。那么怎么平缓一些呢，这时候就要用到加权平均值了，如指数加权平均值。



这条红线怎么计算出来？通过指数加权的公式即：

$$S_t = \begin{cases} Y_1, & t = 1 \\ \beta S_{t-1} + (1 - \beta) Y_t, & t > 1 \end{cases}$$

第一天，第二天，第三天

其中 Y_t 为 t 下的实际值， S_t 为 t 下加权平均后的值， β 为权重值。

上图的红线中， β 为0.9，那么第一天的温度，第二天的温度，第三天的温度计算为 $S_1 = Y_1$

$$S_2 = 0.9S_1 + 0.1Y_2$$

...

上图的红线中， β 为0.9，那么第一天的温度，第二天的温度，第三天的温度计算为 $S_1 = Y_1$

$$S_2 = 0.9S_1 + 0.1Y_2$$

...

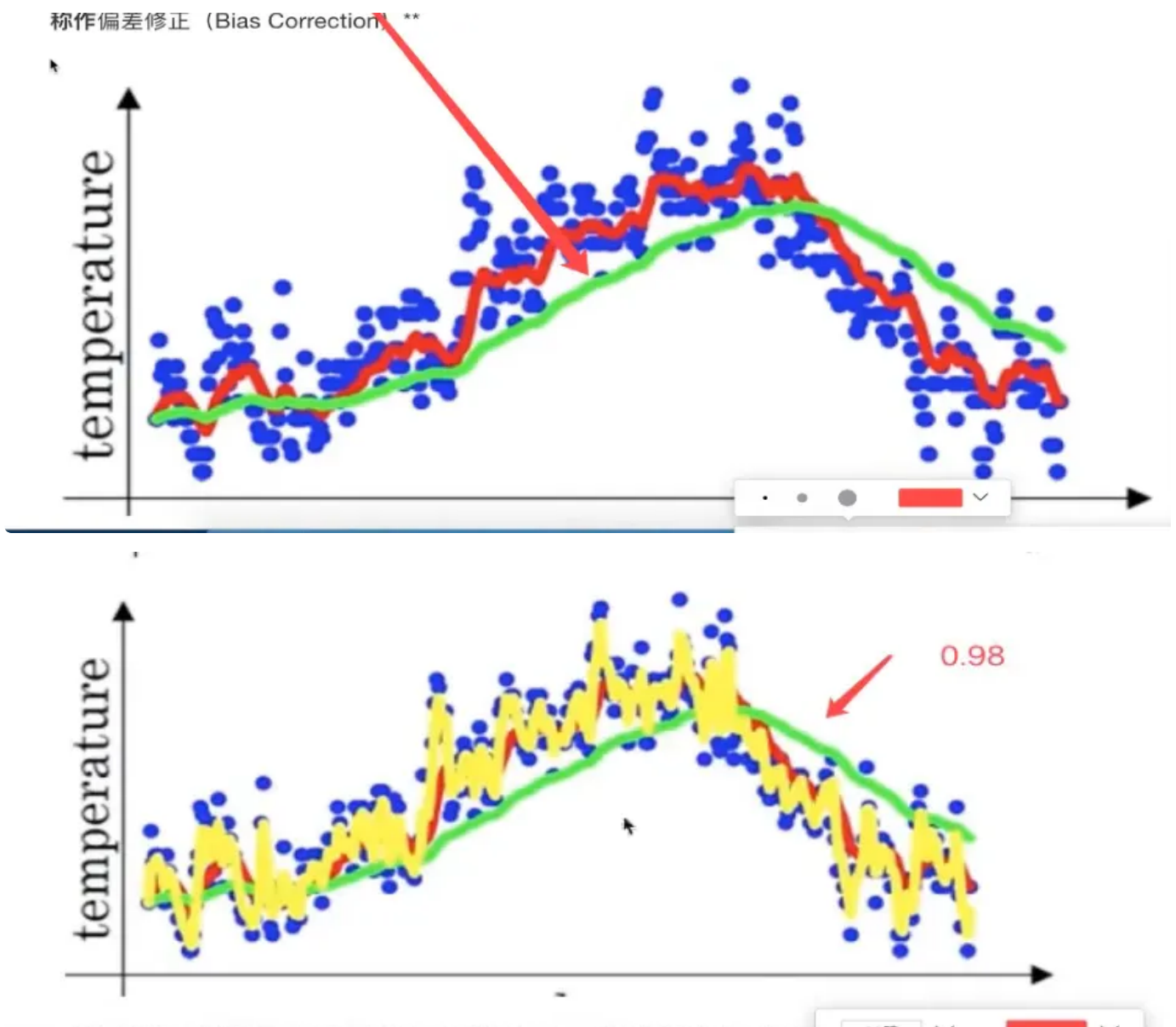
$$S_{99} = 0.9S_{98} + 0.1Y_{99}$$

$$S_{100} = 0.9S_{99} + 0.1Y_{100}$$

...

假设就100天，那么合并的结果 $S_{100} = 0.1Y_{100} + 0.1 * 0.9Y_{99} + 0.1 * (0.9)^2Y_{98} + \dots$

下图中，当取权重值 $\beta=0.98$ 时，可以得到图中更为平滑的绿色曲线。而当取权重值 $\beta=0.5$ 时，得到图中噪点更多的黄色曲线。 β 越大相当于求取平均利用的天数越多，曲线自然就会越平滑而且越滞后。这些系数被称作偏差修正（Bias Correction）**



上述点数据，我们是否可以理解成梯度下降的过程，每一次迭代优化计算出来的梯度值。

动量梯度下降法

动量梯度下降（GDWM）是计算梯度的指数加权平均数，并利用该值来更新参数值。动量梯度下降法的整个过程为：

$$S_{dW}^{[l]} = \beta S_{dW}^{[l]} + (1 - \beta) dW^{[l]}$$

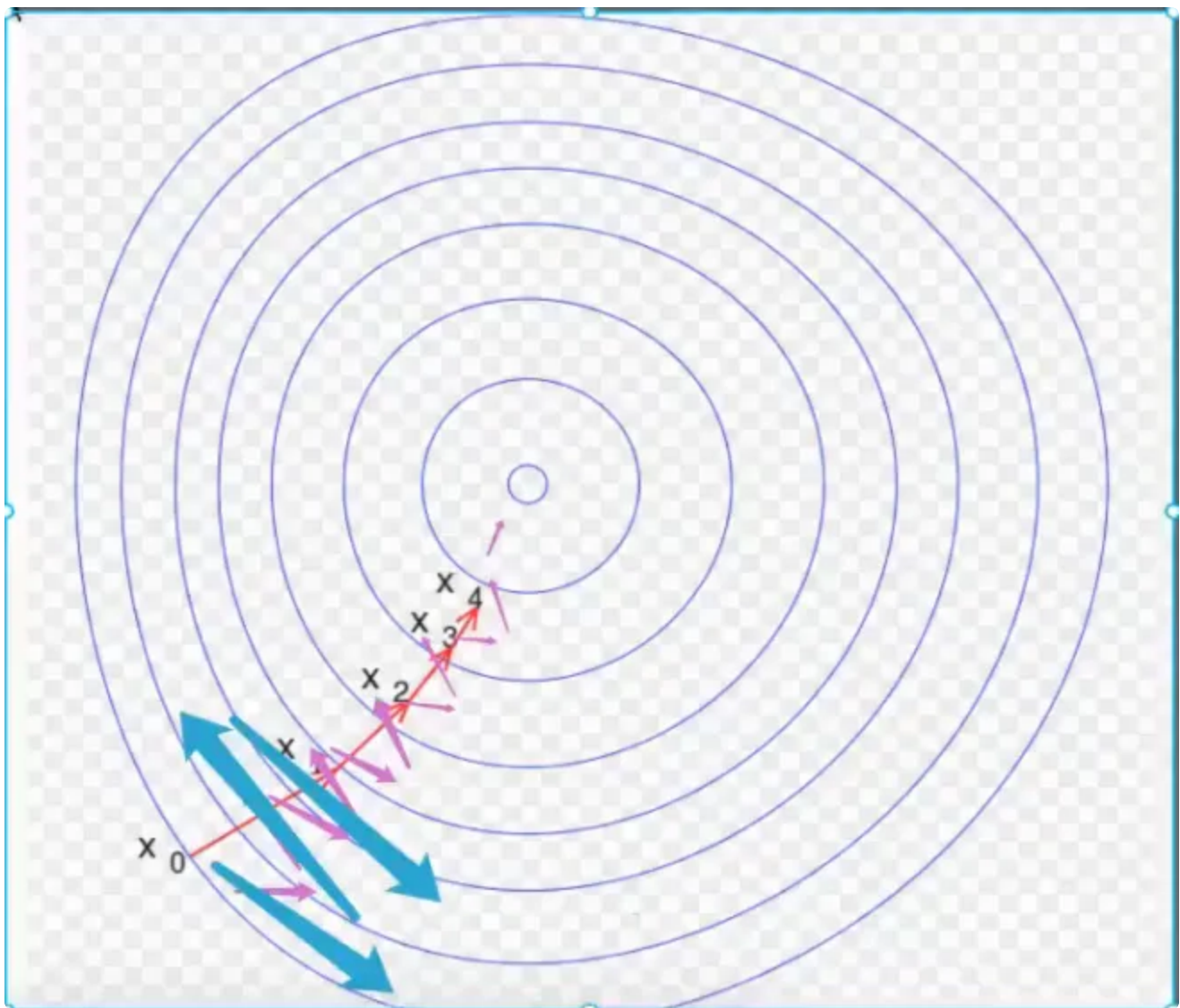
$$S_{db}^{[l]} = \beta S_{db}^{[l]} + (1 - \beta) db^{[l]}$$

$$W^{[l]} := W^{[l]} - \alpha S_{dW}^{[l]}$$

$$b^{[l]} := b^{[l]} - \alpha S_{db}^{[l]}$$

移动平均运算，减小噪点浮动

那么这样梯度下降过程会有什么变化，如图所示：



使用动量梯度下降时，通过累加过去的梯度值来减少抵达最小值路径上的波动，加速了收敛，因此在横轴方向下降得更快，从而得到图中红色或者紫色的曲线。当前后梯度方向一致时，动量梯度下降能够加速收敛；而前后梯度方向不一致时，动量梯度下降能够抑制震荡。

我们可以这样形象的理解，小球在向下运动过程中会有加速度，导致越来越快，由于 β 的存在使得不会一直加速运行。

RMSProp算法

RMSProp (Root Mean Square Prop) 算法是在对梯度进行指数加权平均的基础上，引入平方和平方根。

$$sdw = \beta sdw + (1 - \beta)(dw)^2$$

$$sdb = \beta sdb + (1 - \beta)(db)^2$$

$$w := w - \alpha \frac{dw}{\sqrt{sdw + \epsilon}}$$

$$b := b - \alpha \frac{db}{\sqrt{sdb + \epsilon}}$$

其中 ϵ 是一个非常小的数,防止分母太小导致不稳定,当 dw 或 db 较大时, $(dw)^2, (db)^2$ 会较大,进而 sdw 也会较大,最终使得 $\frac{dw}{\sqrt{sdw + \epsilon}}$ 等结果变得非常小。

最终RMSProp有助于减少抵达最小值路径上的摆动,并允许使用一个更大的学习率 α ,从而加快算法学习速度。

Adam算法

Adam优化算法(自适应矩估计)将Momentum和RMSProp算法结合在一起。

假设用每一个 mini-batch 计算 dW 、 db , 第 t 次迭代时:

$$vdW = \beta_1 vdW + (1 - \beta_1)dW$$

$$vdb = \beta_1 vdb + (1 - \beta_1)db$$

动量梯度结果

$$v_{dW}^{corrected} = \frac{v_{dW}^{[l]}}{1 - (\beta_1)^t}$$

修正

目的: 稍微增大一些调整后的值

$$sdW = \beta_2 sdW + (1 - \beta_2)(dW)^2$$

$$sdb = \beta_2 sdb + (1 - \beta_2)(db)^2$$

RMSProp梯度接

$$s_{dW}^{corrected} = \frac{s_{dW}^{[l]}}{1 - (\beta_2)^t}$$

修正

目的: 稍微增大一些调整后的值

其中 l 为某一层, t 为移动平均第次的值

Adam算法的参数更新：

$$W := W - \alpha \frac{v_{dW}^{corrected}}{\sqrt{s_{dW}^{corrected} + \epsilon}}$$

$$b := b - \alpha \frac{v_{db}^{corrected}}{\sqrt{s_{db}^{corrected} + \epsilon}}$$

从学习率敏感性

- 动量 > RMSProp > Adam

TensorFlow Adam算法API

```
tf.train.AdamOptimizer(learning_rate=0.001, beta1=0.9, beta2=0.999, epsilon=1e-08, name='Adam')
```

Adam优化算法有很多的超参数：

- 学习率 α ：需要尝试一系列的值，来寻找比较合适的
- β_1 ：常用的缺省值为 0.9
- β_2 ：Adam 算法的作者建议为 0.999
- ϵ ：Adam 算法的作者建议为epsilon的默认值1e-8

注： β_1 、 β_2 、 ϵ 通常不需要调试

学习率衰减

如果设置一个固定的学习率 α

在最小值点附近，由于不同的batch中存在一定的噪声，因此不会精确收敛，而是始终在最小值周围一个较大的范围内波动。

如果随着时间慢慢减少学习率 α 的大小，在初期 α 较大时，下降的步长较大，能比较快的速度进行梯度下降；而后期逐步减小 α 的值，即减小步长，有助于算法的收敛，更容易接近最优解。

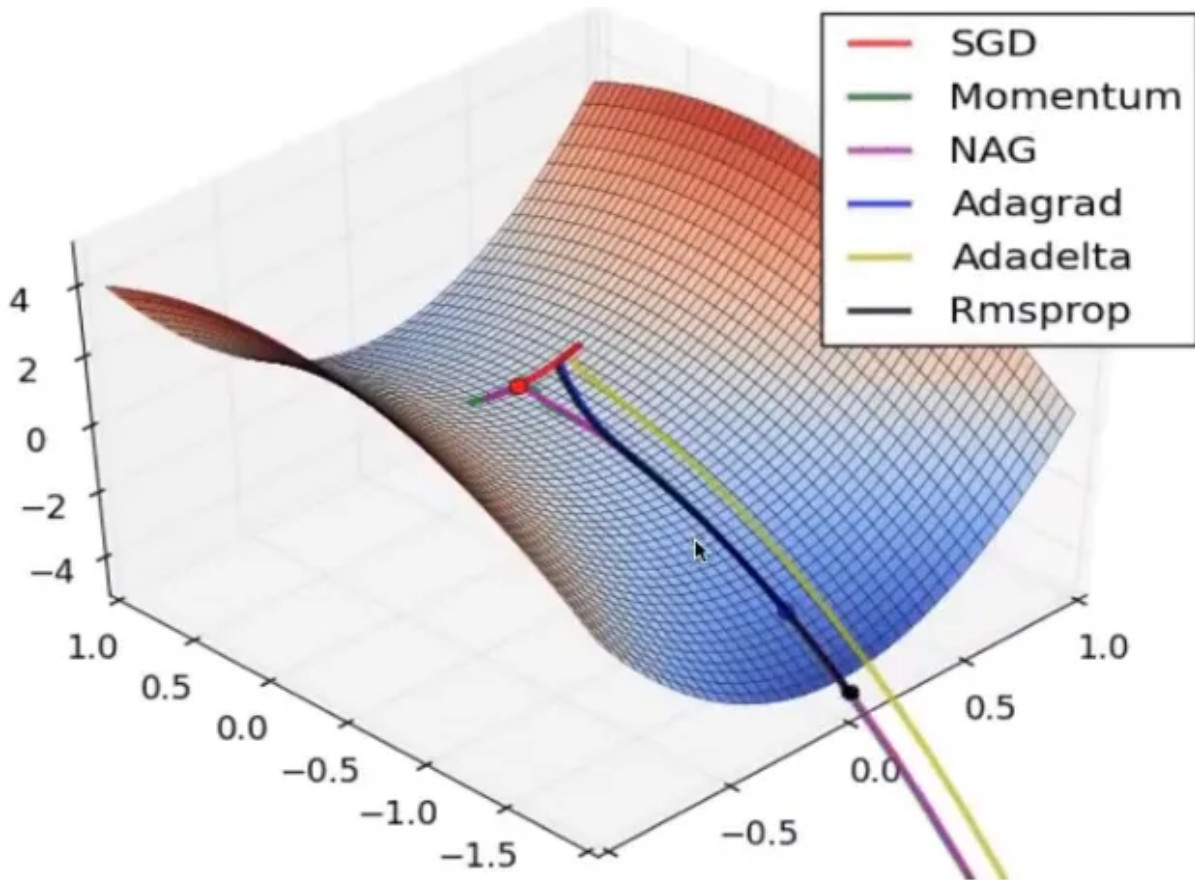
最常用的学习率衰减方法: $\alpha = \frac{1}{1+decay_rate*epoch_num} * \alpha_0$

其中, decay_rate为衰减率(超参数), epoch_num为将所有的训练样本完整过一遍的次数。

还有一种指数衰减

$$\alpha = 0.95^{epoch_num} * \alpha_0$$

对于大型的数据模型, 需要使用这些方式去自动进行学习率衰减。而一些小型网络可以直接手动进行调整那么最后我们来看一张动态图, 表示不同优化的算法的效果图。

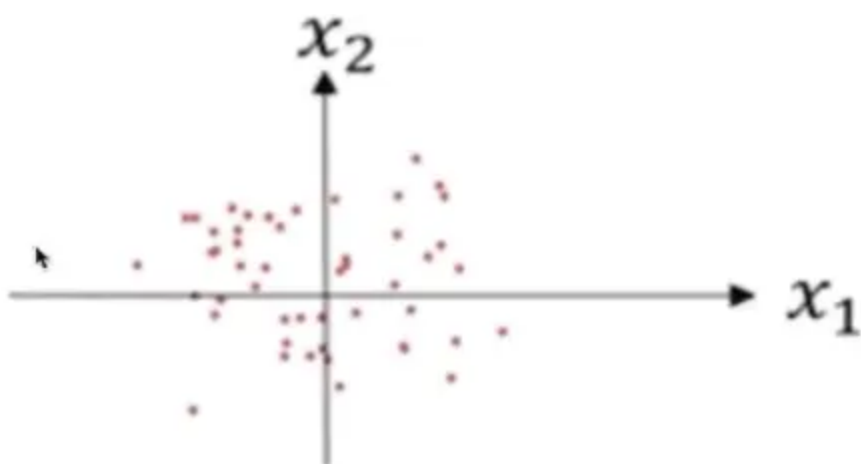
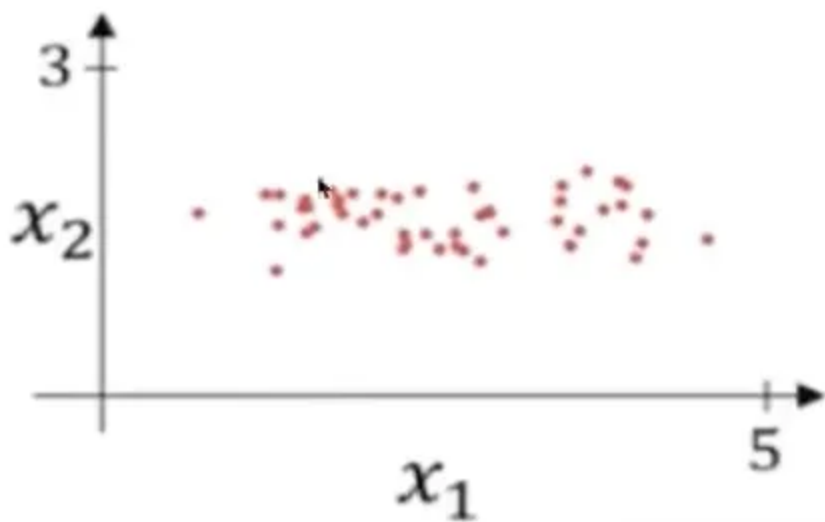


其它非算法优化的方式-标准化输入

对网络输入的特征进行标准化, 能够缓解梯度消失或者梯度爆炸

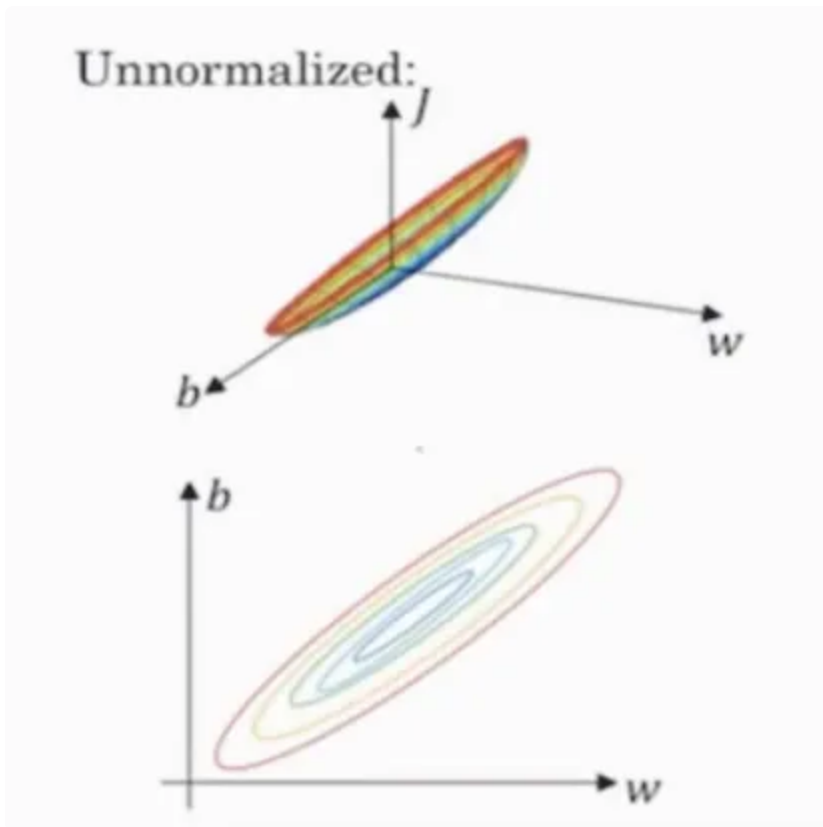
标准化公式: $x = \frac{x - \mu}{\sigma}$

这个公式其实与特征工程中的处理是一样的, μ 为平均值, σ 为标准差。标准化的目的是所有特征的平均值为0, 标准差为1。这属于机器学习基本的内容不过多进行叙述。

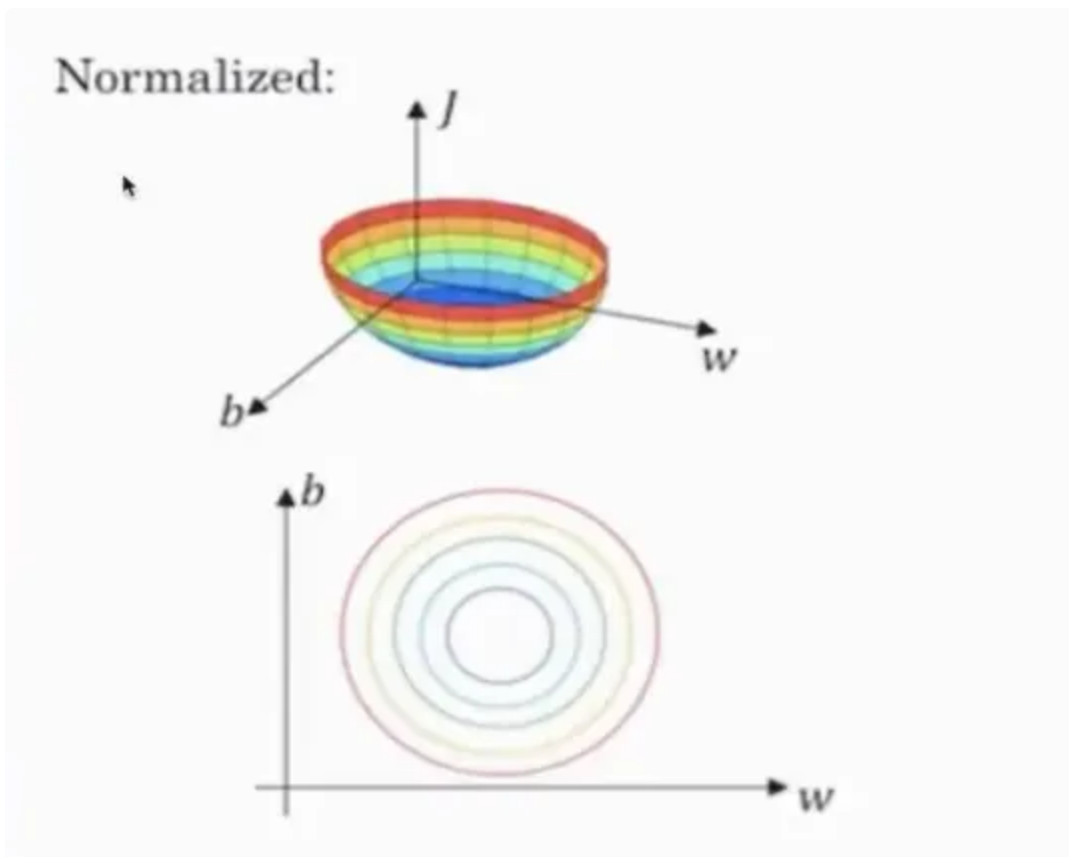


那么这种有什么好处？主要是对于损失函数带来的好处

标准化前的损失函数



标准化后的损失函数



这样的话，对于梯度下降无论从哪个位置开始迭代，都能以相对较少的迭代次数找到全局最优解。可以加速网络的学习。

理解这个原理，其实还是最初的这样的公式 $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$

如果激活函数的输入X近似设置成均值为0，标准方差为1，神经元输出Z的方差就正则化到1了。虽然没有解决梯度消失和爆炸问题，但其在一定程度上确实减缓了梯度消失和爆炸的速度。